Backgammon

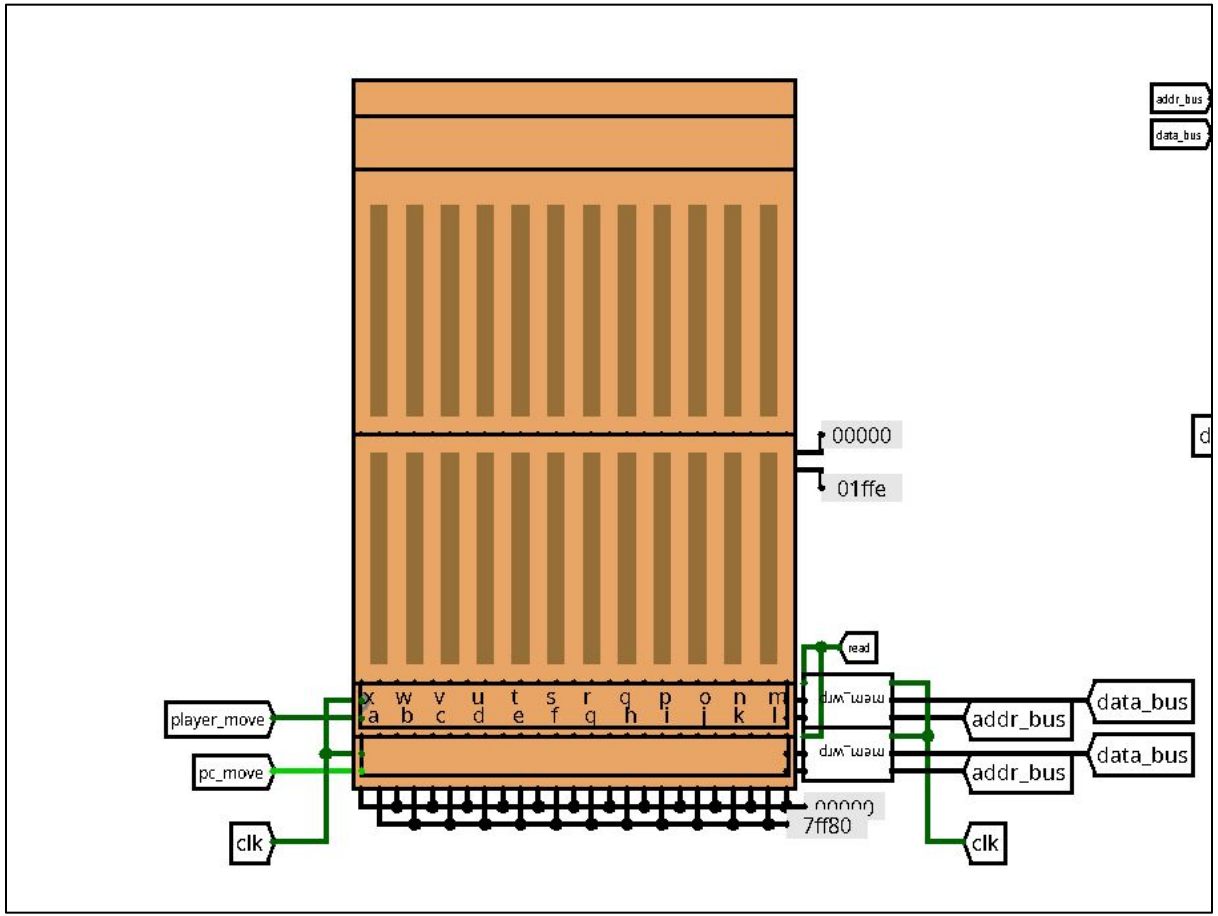
Group 25214
Dedov Dmitry
Starikova Maria
Medvedev Mikhail

Game Rules

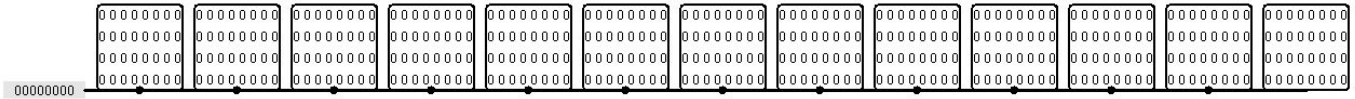
- Goal: Be the first to bear off all 11 checkers
- Start: All checkers are placed on “head” point; Human player makes first move
- Making a move: Roll 2 dice and move your checkers forward by those numbers; Doubles → 4 moves instead of 2; No legal move available → pass manually
- Restrictions: Only 1 checker can be moved from start point per roll (2 if Doubles); Zabor rule - no 6-checkers block unless opponent has started bearing off; No hitting - cannot land a point, which already landed by opponent
- Bearing off: all your 11 checkers are in home; Exact die value, higher value allowed if point is empty and no checkers behind

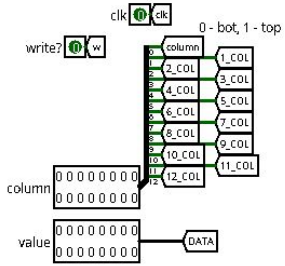
Hardware

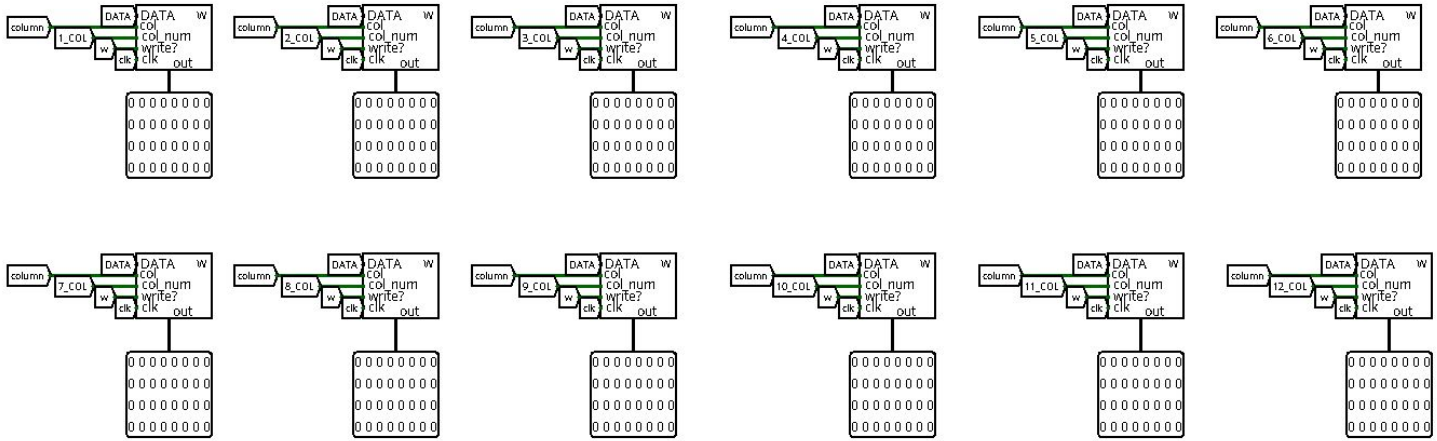
LED Matrix System



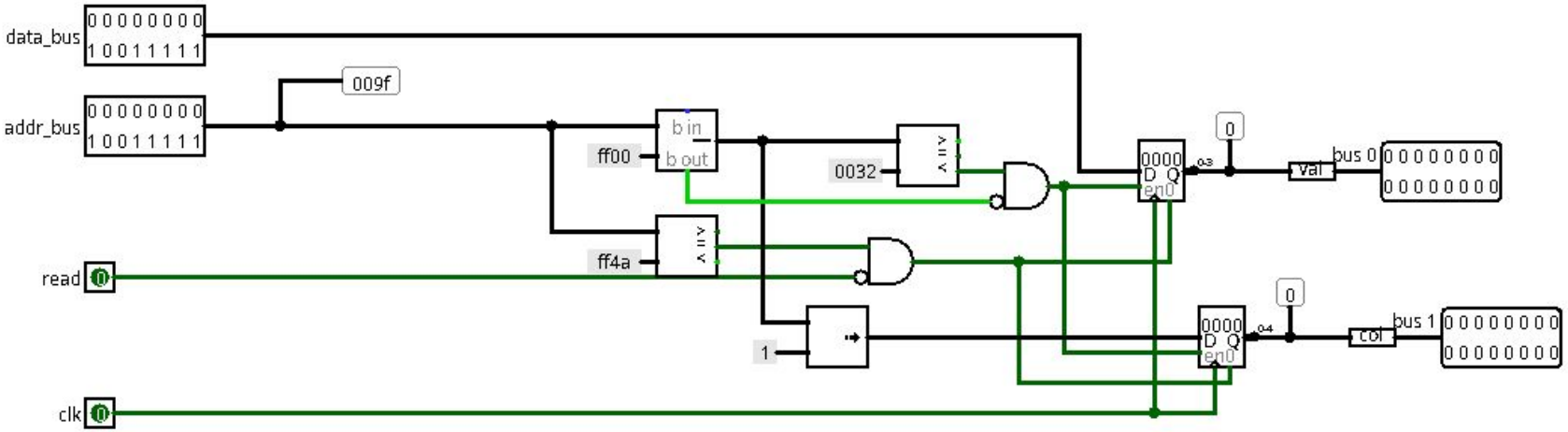
Video Controller



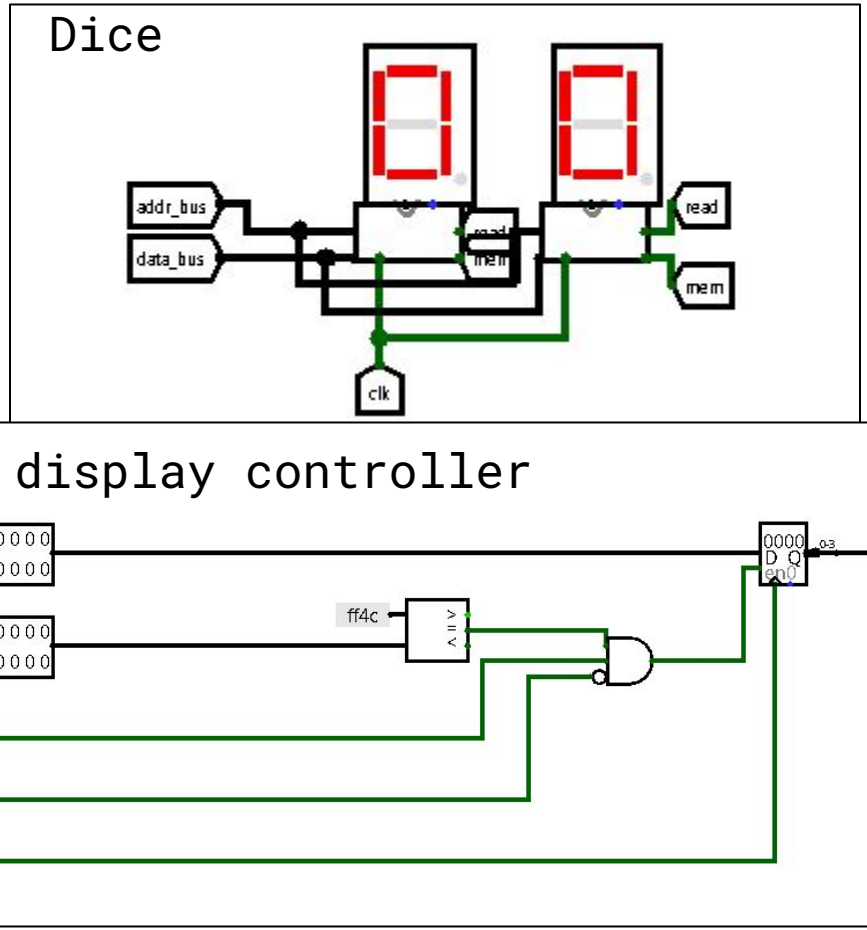




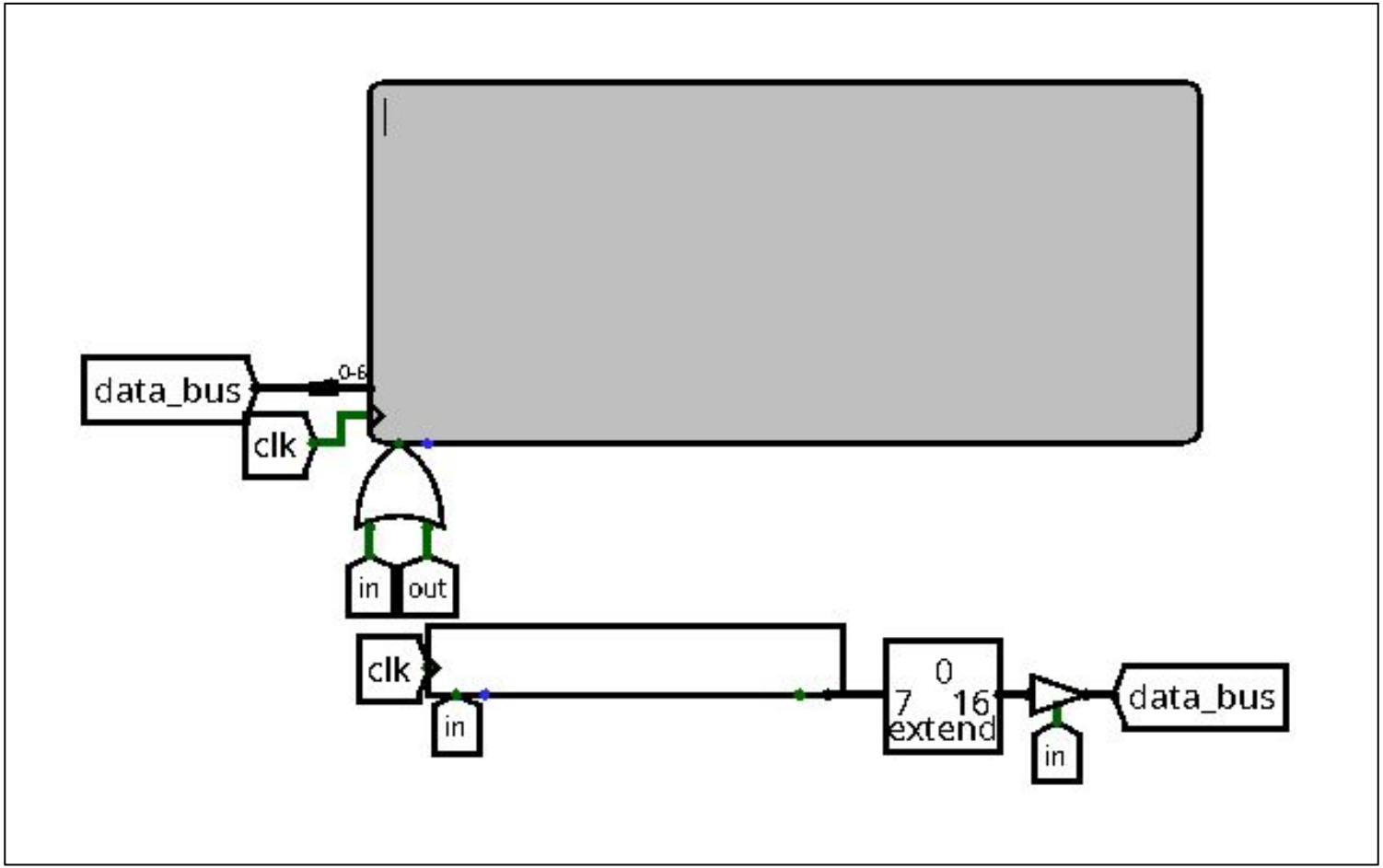
Memory Wrapper



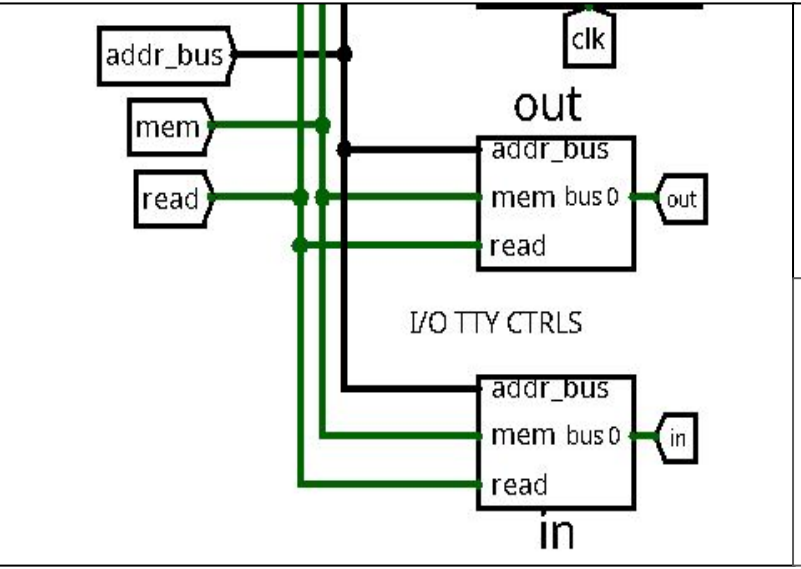
Dice

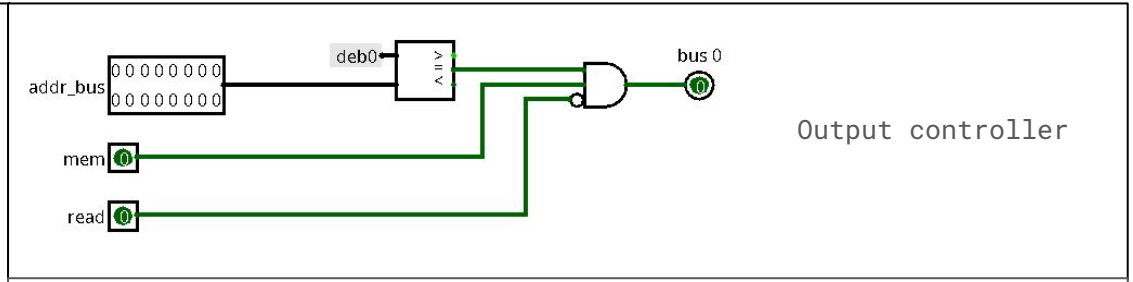

Dice
addr_bus
data_bus
read
mem
clk
Dice display controller
data_bus
addr_bus
mem
read
clk
ff4c
A = V
0000
D Q
en
out
0 0 0 0

Input/Output

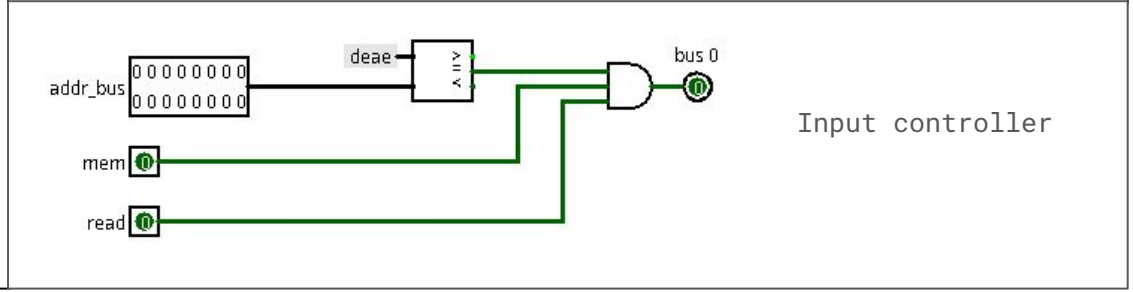


Input/Output controllers





Output controller

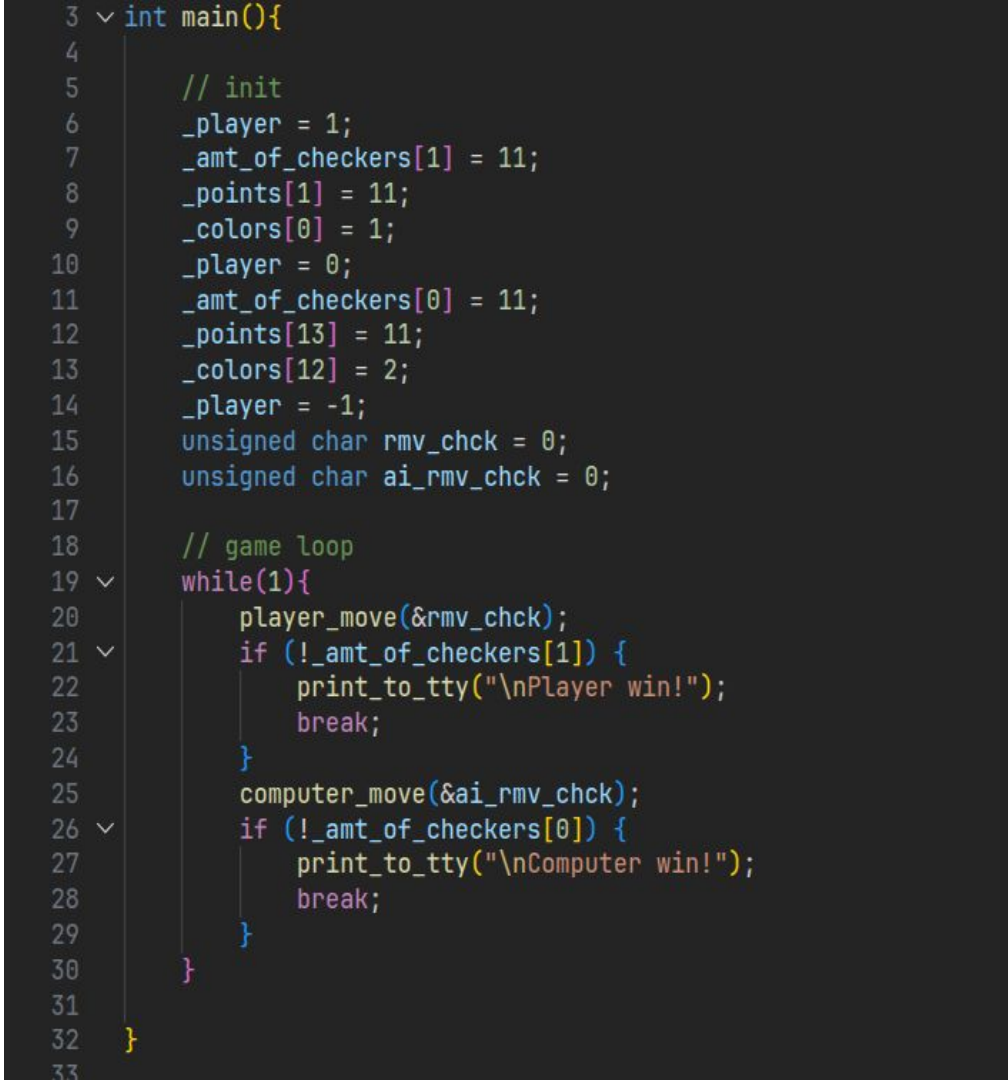


Input controller

Software

Game initialization

Our implementation is a bit simplified, each player has 11 checkers instead of 15



```
int main(){

    // init
    _player = 1;
    _amt_of_checkers[1] = 11;
    _points[1] = 11;
    _colors[0] = 1;
    _player = 0;
    _amt_of_checkers[0] = 11;
    _points[13] = 11;
    _colors[12] = 2;
    _player = -1;
    unsigned char rmv_chck = 0;
    unsigned char ai_rmv_chck = 0;

    // game loop
    while(1){
        player_move(&rmv_chck);
        if (!_amt_of_checkers[1]) {
            print_to_tty("\nPlayer win!");
            break;
        }
        computer_move(&ai_rmv_chck);
        if (!_amt_of_checkers[0]) {
            print_to_tty("\nComputer win!");
            break;
        }
    }

}
```

Player move

```
void player_move(unsigned char* can_remove_checker){
    unsigned char move[2];
    print_to_tty("\n--- Player Turn ---\n");
    randomize();
    _player = 1;

    if (!(*can_remove_checker)){
        if (is_all_in_home()){
            print_to_tty ("\nDBG: all checkers in home\nnow you bear off them");
            *can_remove_checker = 1;
        }
    }

    // _random - physical array, which you can see on displays
    // dice - actual array, which help us with doubles

    unsigned char dice[4];
    int dice_count = 0;
    int head_can_taken = 0;

    char d1 = _random[0];
    char d2 = _random[1];

    if (d1 == d2) {
        print_to_tty("\nRolling doubles!!");
        dice[0] = d1; dice[1] = d1; dice[2] = d1; dice[3] = d1;
        dice_count = 4;
        head_can_taken = 2;
    } else {
        dice[0] = d1; dice[1] = d2;
        dice_count = 2;
        head_can_taken = 1;
    }
```

Random

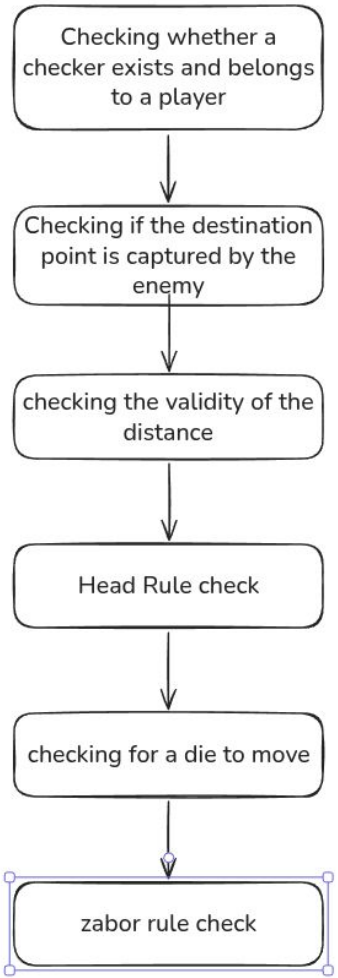
```
unsigned int state = 44257;

unsigned int xorshift16() {
    unsigned int x = state;
    x ^= x << 7;
    x ^= x >> 9;
    x ^= x << 8;
    state = x;
    return x;
}

void randomize(){
    for(int i = 0; i < 2; i++){
        unsigned int val;
        do {
            val = xorshift16() & 0x07;
        } while (val < 1 || val > 6);
        _random[i] = (char)val;
    }
}
```

Player move

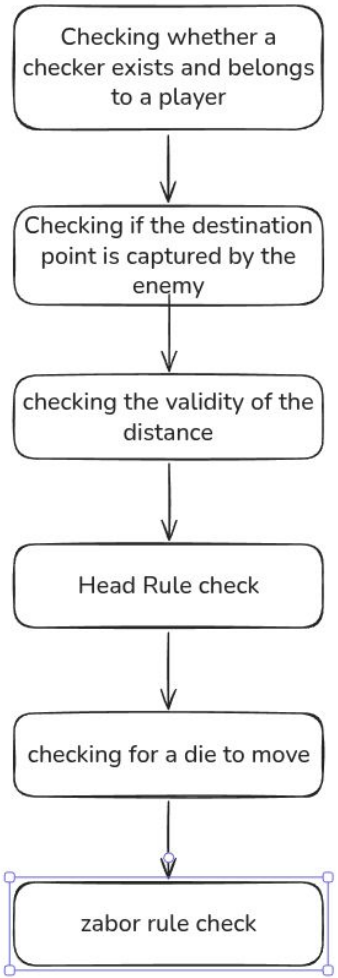
```
68      while (dice_count){
67          print_to_tty("\nMoves left: ");
66          *(char*)TTY = dice_count + '0';
65
64          // * fetch coordinates
63          print_to_tty("\nfrom (z to pass): ");
62          move[0] = getc();
61          if (move[0] == 'z' - 'a') {
60              print_to_tty("\nPassed.\n");
59              break;
58          }
57
56          if (*can_remove_checker) print_to_tty("\nto: (y to bear off) ");
55          else print_to_tty("\nto: ");
54          move[1] = getc();
53
52          if (move[1] == 24){
51              if (!(*can_remove_checker)){
50                  print_to_tty("You cannot bear off now");
49                  continue;
48              } else{
47                  int used_dice = validate_bear_off(move[0], dice, dice_count);
46                  if(!used_dice) continue;
45                  remove_checker(move[0]);
44                  for(int i = 0; i < dice_count; i++){
43                      if (dice[i] == used_dice) {
42                          dice[i] = dice[--dice_count];
41                          break;
40                      }
39                  }
38                  if (d1 != d2) {
37                      if (_random[0] == used_dice) _random[0] = 0;
36                      else if (_random[1] == used_dice) _random[1] = 0;
35                  } else if (dice_count == 2) _random[1] = 0;
34              }
33          } else {
```

Player move

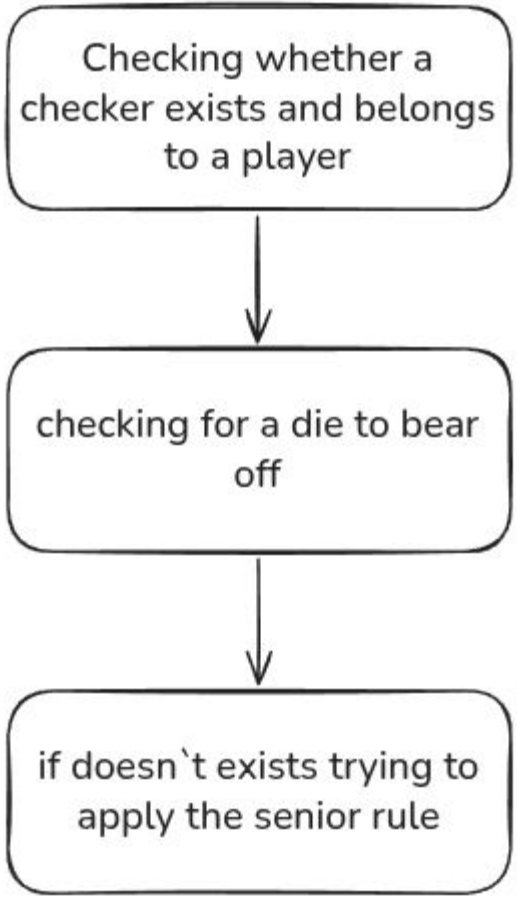
```
32          // normal move, when can_remove_checker down
31          print_to_tty("\nMove validation...");
30
29          if (is_move_valid(move, dice, dice_count, head_can_taken)){
28              move_checker(move);
27              if (!zabor_rule()){
26                  unsigned char undomove[2] = {move[1], move[0]};
25                  move_checker(undomove);
24              } else {
23                  print_to_tty("\nOk...");
22                  if (move[0] == 0) head_can_taken--;
21
20                  int dist = get_dst(move[0], move[1], _player);
19                  for(int i = 0; i < dice_count; i++){
18                      if (dice[i] == dist) {
17                          dice[i] = dice[dice_count - 1];
16                          dice_count--;
15                          break;
14                      }
13                  }
12
11                  if (d1 != d2) {
10                      if (_random[0] == dist) _random[0] = 0;
9                       else if (_random[1] == dist) _random[1] = 0;
8                   } else if (dice_count == 2) _random[1] = 0;
7               }
6           } else {
5               print_to_tty("\nInvalid\n");
4           }
3       }
2   }
1   _player = -1;
```




```c
char is_move_valid(unsigned char* move, unsigned char* dice, int dice_count, int head_can_taken){
    int color = (_player == 1) ? 1 : 2;
    if (_colors[move[0]] != color || !_points[move[0]+1]){
        if (color == 1) print_to_tty("\nErr: Not your checker!\n");
        return 0;
    }

    char opponent = (_player == 1) ? 2 : 1;
    if (_colors[move[1]] == opponent) {
        if (color == 1) print_to_tty("\nErr: Point captured by opponent!\n");
        return 0;
    }

    int dist = get_dst(move[0], move[1], _player);
    if (dist <= 0 || dist > 6) {
        if (color == 1) print_to_tty("\nErr: Invalid dist\n");
        return 0;
    }

    char head_pos = (_player == 1) ? 0 : 12;
    if (move[0] == head_pos && head_can_taken == 0) {
        if (color == 1) print_to_tty("\nErr: Head rule\n");
        return 0;
    }

    char can_move = 0;
    for(int i = 0; i < dice_count; i++){
        if (dice[i] == dist) {
            can_move = 1;
            break;
        }
    }
    if (!can_move) {
        if (color == 1) print_to_tty("\nErr: Dice doesn`t exists\n");
        return 0;
    }

    return 1;
}
```




```
char zabor_rule() {
    if (_amt_of_checkers[!_player] < 11) return 1;

    char opponent = (_player == 1) ? 2 : 1;
    char color = (_player == 1) ? 1 : 2;
    char opp_finish = (opponent == 2) ? 11 : 23;
    for (int i = 0; i < 24; i++) {
        int count = 0;
        for (int j = 0; j < 6; j++) {
            int idx = i + j;
            if (idx >= 24) idx -= 24;

            if (_colors[idx] == color)
                count++;
            else
                break;
        }
        if (count == 6) {
            char dst = get_dst(i+6, opp_finish, 0);
            char found = 0;
            // trying to find opponent chip ahead
            for (int j = 0; j <= dst; j++){
                int curr = i+6+j;
                if (curr >= 24) curr -= 24;
                if (curr >= 24) curr -= 24;
                if (_colors[curr] == opponent){
                    found = 1;
                    break;
                }
            }

            if (!found) {
                print_to_tty("\nErr: Illegal block (6 in row)\n");
                return 0;
            }
        }
        i += count;
    }

    return 1;
}
```




```
char validate_bear_off(unsigned char from, unsigned char* dice, int dice_count) {
    int color = (_player == 1) ? 1 : 2;
    if (_colors[from] != color || _points[from + 1] == 0) {
        print_to_tty("\nErr: Not your checker!\n");
        return 0;
    }

    // get dst
    int dst = (_player == 1) ? (24 - from) : (12 - from);

    // find dice
    for (int i = 0; i < dice_count; i++) {
        if (dice[i] == dst) {
            return dst; // return dice
        }
    }
    // check checkers behind
    char checkers_behind = 0;
    int home_start = (_player == 1) ? 18 : 6;
    int player = _player;
    _player = -1;
    for (int i = home_start; i < from; i++) {
        if (_colors[i] == player && _points[i + 1] > 0) {
            checkers_behind = 1;
            break;
        }
    }
    _player = player;
    // senior dice rule
    if (!checkers_behind) {
        int best_dice = 7;
        for (int i = 0; i < dice_count; i++) {
            if (dice[i] > dst && dice[i] < best_dice) {
                best_dice = dice[i];
            }
        }
        if (best_dice != 7) {
            return best_dice;
        }
    }

    print_to_tty("\nErr: No valid dice to bear off\n");
    return 0;
}
```

Computer move

```
void computer_move(unsigned char* can_remove_checker) {
    unsigned char move[2];
    print_to_tty("\n--- Computer Turn ---\n");
    randomize();
    _player = 0;

    if (!(*can_remove_checker)){
        if (is_all_in_home()){
            print_to_tty ("\nDBG: Computer now can bearing off!");
            *can_remove_checker = 1;
        }
    }

    // _random - physical array, which you can see on displays
    // dice - actual array, which help us with doubles

    unsigned char dice[4];
    int dice_count = 0;
    int head_can_taken = 0;

    char d1 = _random[0];
    char d2 = _random[1];

    // * process double
    if (d1 == d2) {
        print_to_tty("\nRolling doubles!!");
        dice[0] = d1; dice[1] = d1; dice[2] = d1; dice[3] = d1;
        dice_count = 4;
        head_can_taken = 2;
    } else {
        dice[0] = d1; dice[1] = d2;
        dice_count = 2;
        head_can_taken = 1;
    }
```

Computer move

```
27      while (dice_count){
28          int bst_score = -1;
29          char bst_from = -1;
30          char bst_to = -1;
31
32          char best_is_bear_off = 0;
33          int dice_for_bear_off = 0;
34
35          for (int i = 0; i < dice_count; i++) {
36              int curr = dice[i];
37
38              for (char from = 0; from < 24; from++) {
39                  if (_colors[from] != 2) continue;
40
41                  char to = from + curr;
42                  char is_bear_off = 0;
43
44                  if (from >= 12) {
45                      if (to >= 24) to -= 24;
46                  } else {
47                      if (to >= 12) {
48                          if (*can_remove_checker){
49                              to = 24;
50                              is_bear_off = 1;
51                          } else continue;
52                      }
53                  }
54
55                  move[0] = from;
56                  move[1] = to;
57
```

Computer move

```
            if (is_bear_off) {
                int used_dice = validate_bear_off(from, dice, dice_count);
                if (used_dice) {
                    int score = evaluate_move(from, to, 1);
                    if (score > bst_score) {
                        bst_score = score; bst_from = from; bst_to = to;
                        best_is_bear_off = 1;
                        dice_for_bear_off = used_dice;
                    }
                }
            } else {
                if (is_move_valid(move, dice, dice_count, head_can_taken)) {
                    int score = evaluate_move(from, to, 0);
                    move_checker(move);
                    if (zabor_rule()) {
                        if (score > bst_score){
                            bst_score = score; bst_from = from; bst_to = to;
                            best_is_bear_off = 0;
                        }
                    }
                    unsigned char undomove[2] = {to, from};
                    move_checker(undomove);
                }
            }
        }
    }

    if (bst_score == -1) {
        print_to_tty("\nComputer passed.");
        break;
    }

    unsigned char best_move[2] = {bst_from, bst_to};
```

Move Evaluating

We tried to make the check 2 moves ahead, but even it is very slow in some situations.

```
static int evaluate_move(char from, char to, char is_bear_off) {
    int score = 0;

    if (is_bear_off) return 1000;

    if (from == 12) score += 80;

    if (_points[to + 1] == 0) {
        score += 60;

        if (to >= 0 && to <= 5) score += 40;
    }
    else {
        if (_points[to + 1] >= 2) score -= 60;
        else score += 10;
    }

    if (to > 0 && _colors[to - 1] == 2) score += 50;
    if (to < 23 && _colors[to + 1] == 2) score += 50;

    return score;
}
```

Thanks for attention